



DOCUMENTACIÓN PRÁCTICA CATAN

Agente Inteligente para jugar al Catan

Judit Espinoza Cervera y Nerea Aguilar Forés

Contenido

1. Agente implementado	2
1.1 Integración y pruebas iniciales del agente base.....	2
2. Lógica de las funciones implementadas.....	6
on_trade_offer.....	6
on_turn_start	6
on_having_more_than_7_materials_when_thief_is_called	6
on_moving_thief	6
on_turn_end.....	7
on_commerce_phase.....	7
on_build_phase.....	7
on_game_start	8
on_monopoly_card_use	8
on_road_building_card_use	8
on_year_of_plenty_card_use.....	8
3. Hiperparámetros y parámetros del agente base	9
4. Versión genética del agente	10
5. Diseño del algoritmo genético.....	11
Fitness	12
6. Hiperparámetros del algoritmo genético	12
7. Resultados del entrenamiento genético	13
8. Validación final y mejor individuo	14
9. Evaluación comparativa	15
Evaluación con rivales mixtos	15
Evaluación contra agentes estándar sin RandomAgent	15
Evaluación contra 10000 partidas con RandomAgent	15
10. Uso de herramientas de IA	16
11. Conclusiones	17

Documentación del agente NereaJudit y algoritmo genético

1. Agente implementado

El agente desarrollado por nuestro grupo corresponde al archivo *Agents/NereaJuditAgent.py*. Este agente parte de una lógica heurística, es decir, usa decisiones programadas manualmente a partir de prioridades del juego de Catan. La idea principal es conseguir recursos útiles, construir estructuras que den puntos de victoria y usar el ladrón o las cartas de desarrollo para mejorar la posición del agente durante la partida.

La lógica principal del agente base es propia. Aun así, en algunos criterios concretos se usó apoyo de IA para decidir prioridades estratégicas, por ejemplo, en el orden de descarte, en la prioridad de construcción y en el uso de algunas cartas de desarrollo. No se copiaron métodos completos de otros agentes del repositorio, aunque si se tuvo en cuenta la estructura general que exige *AgentInterface*.

Además del agente base, se creó una versión parametrizable llamada *Agents/GeneticNereaJuditAgent.py*. Esta versión está basada en la lógica del agente *NereaJuditAgent.py*, pero transforma sus prioridades fijas en genes numéricos para que puedan ser entrenadas mediante un algoritmo genético.

1.1 Integración y pruebas iniciales del agente base

El agente *NereaJuditAgent* fue probado dentro del simulador usando el archivo *run_execution.py*. En estas pruebas se configuró una partida con varios agentes del repositorio, incluyendo *RandomAgent*, *AdrianHerasAgent* y *AlexPastorAgent*.

El objetivo de estas pruebas iniciales no era todavía entrenar el algoritmo genético, sino comprobar que el agente base funcionaba dentro del simulador y que era capaz de tomar decisiones durante una partida completa. El fitness usado en estas primeras pruebas era sencillo: valía 1 si el agente ganaba la partida y 0 si no la ganaba.

En una partida de ejemplo se pudieron ver varias acciones del agente. Aquí se muestra un fragmento amplio de la salida:

--- Partida 1 ---

Compro carta de desarrollo

Juego carta de soldado

Juego carta de soldado al inicio

Construyo una carretera entre: 10 y 2

Compro carta de desarrollo

Juego carta de construcción de carreteras

Compro carta de desarrollo

Juego carta de soldado

Juego carta de soldado al inicio

Compro carta de desarrollo

Construyo una carretera entre: 2 y 1

Compro carta de desarrollo

Juego carta de soldado

Juego carta de soldado al inicio

Construyo una carretera entre: 1 y 0

Construyo un pueblo en: 0

Compro carta de desarrollo

Juego carta de soldado

Juego carta de soldado al inicio

Compro carta de desarrollo

Juego carta de construcción de carreteras

Compro carta de desarrollo

Construyo una carretera entre: 8 y 0

Juego carta de monopolio

Juego carta de monopolio

Compro carta de desarrollo

Juego carta de soldado

Juego carta de soldado al inicio

Compro carta de desarrollo

Compro carta de desarrollo

Juego carta de soldado

Juego carta de soldado al inicio

Compro carta de desarrollo

Puntuación de fitness calculada para el agente NereaJuditAgent: 1

En esta salida se ve que el agente no solo construye, sino que también compra cartas, juega soldados, usa monopolio, construye carreteras y llega a construir un pueblo. Esto demuestra que se activan varias de las funciones implementadas y que el agente no se queda parado durante la partida.

Después se hizo una primera prueba de 10 partidas. En esta prueba se registró el fitness de cada partida:

--- Partida 1 ---

Puntuación de fitness calculada para el agente NereaJuditAgent: 0

--- Partida 2 ---

Puntuación de fitness calculada para el agente NereaJuditAgent: 1

--- Partida 3 ---

Puntuación de fitness calculada para el agente NereaJuditAgent: 1

--- Partida 4 ---

Puntuación de fitness calculada para el agente NereaJuditAgent: 0

--- Partida 5 ---

Puntuación de fitness calculada para el agente NereaJuditAgent: 0

--- Partida 6 ---

Puntuación de fitness calculada para el agente NereaJuditAgent: 1

--- Partida 7 ---

Puntuación de fitness calculada para el agente NereaJuditAgent: 1

--- Partida 8 ---

Puntuación de fitness calculada para el agente NereaJuditAgent: 1

--- Partida 9 ---

Puntuación de fitness calculada para el agente NereaJuditAgent: 0

--- Partida 10 ---

Puntuación de fitness calculada para el agente NereaJuditAgent: 1

===== RESULTADOS FINALES =====

Partidas jugadas: 10

Victorias de NereaJuditAgent: 6

Tasa de victoria: 0.60

Estos resultados mostraron que el agente podía competir de forma estable en una muestra pequeña, ya que gano más de la mitad de las partidas ejecutadas. Aun así, como Catan depende mucho del azar de dados, tablero y cartas, una prueba de 10 partidas puede ser demasiado corta. Por eso se hizo otra prueba más amplia con 50 partidas:

Partidas jugadas: 50

Victorias de NereaJuditAgent: 22

Tasa de victoria: 0.44

La tasa bajó respecto a la prueba de 10 partidas, algo esperable al aumentar el número de simulaciones y reducir el peso de partidas aisladas favorables. En conjunto, estos resultados indican que el agente base es capaz de competir contra otros agentes del simulador. Aunque todavía existen muchas posibilidades de mejora estratégica, el comportamiento general del agente puede considerarse estable y funcional.

Estas pruebas iniciales son importantes porque sirven como punto de partida antes de crear la versión genética. Primero comprobamos que la lógica manual funcionaba y después se transformaron varias de sus prioridades en genes entrenables.

2. Lógica de las funciones implementadas

on_trade_offer

Esta función decide si se acepta o se rechaza una oferta de comercio de otro jugador.

Primero comprueba si el agente puede pagar lo que el rival pide. Si no tiene recursos suficientes, rechaza la oferta. Después rechaza intercambios en los que tenga que entregar cereal o mineral, porque son recursos importantes para ciudades y cartas de desarrollo. En cambio, acepta ofertas que le den cereal o mineral. También acepta si recibe un recurso básico que no tiene, como arcilla, madera o lana.

La lógica es propia, con apoyo de IA para decidir que intercambios suelen ser más convenientes.

on_turn_start

Se ejecuta al inicio del turno, antes de tirar los dados. El agente revisa si tiene cartas de desarrollo disponibles y, si posee una carta de soldado/caballero, intenta jugarla para mover el ladrón antes de la tirada.

La idea es usar el ladrón para bloquear a un rival o mejorar la situación propia antes de que se produzcan recursos. Esta parte se marcó en el código como apoyada por IA para entender mejor el momento adecuado de uso de las cartas.

on_having_more_than_7_materials_when_thief_is_called

Cuando sale un 7 y el agente tiene más de 7 recursos, debe descartar la mitad redondeando hacia abajo. La función crea una mano de descarte y elimina recursos siguiendo este orden:

1. Lana
2. Arcilla
3. Madera
4. Cereal
5. Mineral

El criterio conserva cereal y mineral hasta el final porque son importantes para construir ciudades y comprar cartas de desarrollo. Esta prioridad de descarte fue una de las partes consultadas con IA.

on_moving_thief

Cuando el agente debe mover el ladrón, recorre los terrenos del tablero y busca el primer terreno sin ladrón que tenga un jugador rival en alguno de sus nodos adyacentes. Si lo encuentra, mueve el ladrón a ese terreno y selecciona a ese rival como objetivo del robo.

Es una heurística sencilla: prioriza bloquear y robar a cualquier rival disponible. No calcula todavía el terreno optimo según probabilidad o puntos del rival, pero evita mover el ladrón sin sentido.

on_turn_end

Al final del turno, el agente intenta jugar cartas de desarrollo siguiendo este orden:

1. Soldado/caballero
2. Construcción de carreteras
3. Año de abundancia
4. Monopolio

La lógica busca aprovechar las cartas antes de terminar el turno. El orden da prioridad primero al control del ladrón, después a la expansión por carreteras y finalmente a cartas de obtención de recursos.

on_commerce_phase

Durante la fase de comercio, el agente identifica el recurso que más tiene y lo ofrece si tiene más de una unidad. Después pide el primer recurso que le falte siguiendo este orden:

1. Cereal
2. Mineral
3. Arcilla
4. Madera
5. Lana

Si no tiene recursos suficientes para ofrecer o no le falta ningún recurso relevante, no comercia. La función devuelve un diccionario para comercio con banco o puerto.

on_build_phase

En la fase de construcción, el agente intenta construir siguiendo esta prioridad:

1. Ciudad
2. Pueblo
3. Carta de desarrollo
4. Carretera

La ciudad se prioriza porque aumenta puntos y producción de recursos. El pueblo también da puntos y amplía la producción. La carta de desarrollo puede aportar soldados, puntos o recursos. La carretera queda al final porque por sí sola no da puntos, aunque permite expandirse.

Esta prioridad aparece en el código como una decisión apoyada por IA.

on_game_start

En la colocación inicial, el agente revisa todos los nodos válidos y calcula una puntuación para cada nodo sumando la probabilidad de los terrenos adyacentes. Elige el nodo con mayor puntuación total y construye una carretera hacia el primer nodo adyacente disponible.

Esta lógica es propia y usa información del tablero para colocar el primer pueblo en una zona con buena probabilidad de producir recursos.

on_monopoly_card_use

Cuando juega una carta de monopolio, el agente pide el primer recurso cuya cantidad en mano sea menor que 2. La prioridad es:

1. Cereal
2. Mineral
3. Arcilla
4. Madera
5. Lana

Si todos los recursos están al menos en 2 unidades, pide cereal por defecto. El criterio busca cubrir carencias de la mano. En el código se indica que se consultó IA para decidir este criterio.

on_road_building_card_use

Cuando usa la carta de construcción de carreteras, el agente obtiene las carreteras validas disponibles y selecciona las dos primeras. Si no hay al menos dos opciones validas, no realiza la acción.

Es una heurística sencilla y propia. No optimiza todavía por camino más largo o acceso a puertos, pero permite aprovechar la carta si existen posiciones validas.

on_year_of_plenty_card_use

Cuando usa la carta de año de abundancia, el agente ordena sus recursos de menor a mayor cantidad y pide los dos recursos que menos tiene. Esto equilibra la mano y aumenta la probabilidad de poder construir en turnos siguientes.

La lógica es propia y está orientada a reducir carencias.

3. Hiperparámetros y parámetros del agente base

El agente base no tiene hiperparámetros entrenables como tal, pero si utiliza parámetros de decisión fijos. Estos valores controlan su comportamiento y por eso se pueden explicar cómo hiperparámetros manuales.

Parámetro	Valor usado	Función	Impacto esperado
Prioridad de construcción	Ciudad > Pueblo > Carta > Carretera	<i>on_build_phase</i>	Favorece puntos y producción antes que expansión.
Recursos protegidos	Cereal y mineral	<i>on_trade_offer</i>	Conserva recursos necesarios para ciudades y cartas.
Recursos preferidos	Cereal y mineral	<i>on_trade_offer, on_commerce_phase</i>	Mejora la capacidad de construir ciudades y cartas.
Orden de descarte	Lana > Arcilla > Madera > Cereal > Mineral	<i>on_having_more_than_7_materials_when_thief_is_called</i>	Protege recursos de alto valor.

Parámetro	Valor usado	Función	Impacto esperado
Umbral para comerciar	Mas de 1 unidad	<i>on_commerce_phase</i>	Evita quedarse sin un recurso al comerciar.
Umbral de monopolio	Recurso con menos de 2 unidades	<i>on_monopoly_card_use</i>	Intenta compensar escasez.
Colocación inicial	Suma de probabilidades adyacentes	<i>on_game_start</i>	Favorece nodos productivos.
Selección de carreteras	Primeras opciones validas	<i>on_build_phase, on_road_building_card_use</i>	Es rápido, aunque no siempre optimo.

4. Versión genética del agente

Para cumplir la parte principal de la práctica, se creó el archivo `Agents/GeneticNereaJuditAgent.py`. Este agente está basado en la lógica del agente base, pero cambia las prioridades fijas por genes numéricos.

La idea es que un individuo del algoritmo genético no es un agente nuevo escrito desde cero, sino una configuración distinta de pesos. Por ejemplo, un individuo puede valorar mucho construir pueblos, mientras que otro puede valorar más comprar cartas de desarrollo.

Los genes usados son:

Gen	Significado
<i>build_city</i>	Importancia de construir ciudad.

Gen	Significado
<i>build_town</i>	Importancia de construir pueblo.
<i>build_card</i>	Importancia de comprar carta de desarrollo.
<i>build_road</i>	Importancia de construir carretera.
<i>resource_cereal</i>	Valor estratégico del cereal.
<i>resource_mineral</i>	Valor estratégico del mineral.
<i>resource_clay</i>	Valor estratégico de la arcilla.
<i>resource_wood</i>	Valor estratégico de la madera.
<i>resource_wool</i>	Valor estratégico de la lana.
<i>trade_min_surplus</i>	Cantidad mínima para ofrecer un recurso.
<i>monopoly_threshold</i>	Cantidad mínima deseada antes de pedir un recurso con monopolio.
<i>thief_aggression</i>	Penalización por poner el ladrón en un terreno donde también tenemos construcciones propias.

Con esta representación, la fase de construcción ya no depende solo de un orden fijo. El agente genera las acciones posibles, calcula una puntuación para cada una y ejecuta la mejor. La puntuación mezcla los genes de construcción con la calidad del nodo del tablero.

5. Diseño del algoritmo genético

El algoritmo genético se implementó en *train_genetic_nerea_judit.py*. Su objetivo es encontrar una combinación de genes que haga jugar mejor al agente.

El proceso es:

1. Crear una población inicial con genes aleatorios y también incluir la configuración base.
2. Evaluar cada individuo jugando partidas completas.
3. Calcular un fitness medio para cada individuo.

4. Ordenar los individuos según su fitness.
5. Mantener los mejores mediante elitismo.
6. Seleccionar padres mediante torneo.
7. Crear hijos usando cruce uniforme.
8. Aplicar mutación a algunos genes.
9. Repetir durante varias generaciones.
10. Validar los mejores candidatos en partidas adicionales.
11. Guardar el mejor individuo final.

Fitness

La función de fitness usada es:

$$fitness = puntos * 10 + bonus_victoria + bonus_por_ganar_antes$$

Si el agente gana, recibe un bonus de 100 puntos. Además, si gana antes del máximo de rondas, recibe un pequeño bonus por rapidez. Así se premia ganar, conseguir puntos y terminar antes.

6. Hiperparámetros del algoritmo genético

En el entrenamiento final se usaron estos valores:

Hiperparámetro	Valor usado	Impacto
Tamaño de población	10 individuos	Mas población explora más estrategias, pero tarda más.
Numero de generaciones	5	Mas generaciones permiten evolucionar más, pero aumentan el tiempo.
Partidas por individuo	3	Reduce un poco el azar frente a evaluar una sola partida.
Partidas de validación	30	Permiten elegir un mejor individuo más fiable al final.
Máximo de rondas	120	Evita partidas demasiado largas.
Elite	2 individuos	Conserva las mejores soluciones entre generaciones.

Hiperparámetro	Valor usado	Impacto
Tasa de mutación	0.15	Da diversidad a la población.
Intensidad de mutación	0.20	Controla cuanto cambia un gen cuando muta.
Selección	Torneo	Es simple y estable.
Cruce	Uniforme	Cada gen del hijo viene de uno de los dos padres.
Rivales	Mixtos	Incluye random y agentes estándar para evitar entrenar solo contra un tipo de rival.

7. Resultados del entrenamiento genético

El entrenamiento final se ejecutó con población 10, 5 generaciones, 3 partidas por individuo y rivales mixtos. El registro de fitness medio y máximo por generación se guardó en *docs/genetic_training_final.csv*.

Generación	Fitness medio	Fitness máximo	Tasa victoria mejor	Puntos medios mejor
0	219.33	299.67	1.00	10.00
1	223.27	300.67	1.00	10.00
2	229.30	295.67	1.00	10.00
3	192.50	285.33	1.00	10.00
4	204.03	293.67	1.00	10.00

Durante el entrenamiento, el mejor individuo parecía ser el de la generación 1. Sin embargo, como Catan tiene mucho azar, se decidió no quedarse solo con ese resultado. Para evitar elegir un individuo que hubiese tenido suerte, se añadió una fase de validación final con 30 partidas.

8. Validación final y mejor individuo

En la validación final se compararon la configuración base y los mejores individuos encontrados durante el entrenamiento. El mejor resultado fue *best_generation_2*.

Candidato	Generación	Fitness	Tasa victoria	Puntos medios	Rondas medias
<i>best_generation_2</i>	2	245.17	0.80	8.83	23.63
<i>best_generation_3</i>	3	222.93	0.73	8.43	28.77
<i>base_genes</i>	Base	205.70	0.67	8.03	29.73
<i>best_training_overall</i>	1	196.03	0.63	7.97	33.50
<i>best_generation_4</i>	4	188.43	0.60	7.43	28.40

El mejor individuo final fue:

build_city = 8.0

build_town = 10.086197886799686

build_card = 0.9904156891971647

build_road = 3.0

resource_cereal = 4.0

resource_mineral = 4.0

resource_clay = 0.6692219604169143

resource_wood = 3.493557450034003

resource_wool = 2.0

trade_min_surplus = 1.0

monopoly_threshold = 2.636987157934633

thief_aggression = 4.0

Este individuo prioriza mucho la construcción de pueblos, mantiene alta la importancia de ciudades y baja bastante el peso de las cartas de desarrollo. También permite comerciar con menos excedente, porque *trade_min_surplus* queda en 1. Esto puede

ayudar a conseguir recursos antes, aunque también implica comerciar de forma más agresiva.

9. Evaluación comparativa

Después de elegir el mejor individuo final, se hizo una evaluación independiente para comparar la configuración base con el agente genético entrenado.

Esta evaluación se implementó en el archivo *evaluate_genetic_nerea_judit.py*. Se separó del entrenamiento para comprobar si el mejor individuo encontrado funcionaba bien en partidas nuevas y no solo en las partidas usadas durante el proceso genético.

Evaluación con rivales mixtos

Archivo: *docs/genetic_evaluation_summary.csv*

Configuración	Partidas	Victorias	Tasa victoria	Puntos medios	Fitness medio
Base	30	23	0.77	8.63	228.77
Genético final	30	25	0.83	9.23	248.70

En esta prueba, el agente genético final supera a la configuración base tanto en victorias como en puntos medios y fitness.

Evaluación contra agentes estándar sin RandomAgent

Archivo: *docs/genetic_evaluation_standard_summary.csv*

Configuración	Partidas	Victorias	Tasa victoria	Puntos medios	Fitness medio
Base	30	17	0.57	7.47	178.60
Genético final	30	19	0.63	8.03	194.80

Esta prueba es importante porque el criterio de evaluación indica que se valora ganar más del 25% contra agentes estándar proporcionados, sin contar random. El agente genético final obtiene una tasa de victoria del 63%, por encima de ese umbral.

Evaluación contra 10000 partidas con RandomAgent

También se realizó la prueba opcional indicada en los criterios de evaluación: comprobar si el agente gana más de la mitad de 10000 partidas contra otros 3 agentes random. Para esto se creó el script *benchmark_random_nerea_judit.py*, que ejecuta

muchas partidas del agente genético final contra tres *RandomAgent* y guarda un resumen en CSV.

Archivo: *docs/random_10000_summary.csv*

Partidas	Victorias	Tasa victoria	Puntos medios	Rondas medias
10000	9998	0.9998	10.04	25.88

El resultado supera claramente el umbral del 50% indicado en los criterios, ya que el agente genético final gano 9998 de 10000 partidas contra tres agentes random. Esto muestra que contra rivales aleatorios el comportamiento del agente es muy estable.

10. Uso de herramientas de IA

Se ha usado ChatGPT como apoyo durante el desarrollo. El uso principal fue:

- ayudar a organizar la documentación
- proponer una forma clara de representar el cromosoma
- apoyar en el diseño de la función de fitness
- comparar opciones de selección, cruce y mutación
- revisar resultados y decidir añadir una fase de validación final
- ayudar a crear el script de evaluación comparativa entre la configuración base y el agente genético
- ayudar a preparar el benchmark opcional de 10000 partidas contra tres agentes random
- redactar comentarios explicativos en el código

En los archivos de código también se han dejado comentarios marcados con *USO IA* en algunas partes concretas donde se recibió apoyo, por ejemplo, en la elección de genes, la función de fitness, la selección por torneo, la mutación, la validación final y la evaluación comparativa.

Las decisiones finales se adaptaron al proyecto y al simulador. La lógica base del agente fue implementada por el grupo y la versión genética se construyó a partir de esa lógica.

11. Conclusión

El agente base *NereaJuditAgent* funciona correctamente como agente heurístico y toma decisiones en muchas fases del juego. A partir de esa lógica se creó *GeneticNereaJuditAgent*, una versión parametrizable que permite entrenar prioridades mediante un algoritmo genético.

El algoritmo genético implementado incluye población, fitness, selección por torneo, cruce uniforme, mutación, elitismo y validación final. Tras el entrenamiento y la validación, el mejor individuo final supero a la configuración base en las evaluaciones realizadas.